

```
In [372... #Importing all the required libraries and data for Time Series Analysis

from os.path import basename, exists

def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve

        local, _ = urlretrieve(url, filename)
        print("Downloaded " + local)

download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/thinkst
download("https://github.com/AllenDowney/ThinkStats2/raw/master/code/thinkpl
```

```
In [374... # Import additional functions required to get Correlation and AutoCorrelatio

from __future__ import print_function, division

import pandas
import numpy as np
import statsmodels.formula.api as smf
import statsmodels.tsa.stattools as smtsa

import matplotlib.pyplot as pyplot

import thinkplot
import thinkstats2

FORMATS = ['png']
```

```
In [376... transactions = pd.read_csv("mj-clean.csv", parse_dates=[5])
transactions.head()
```

Out [376...

	city	state	price	amount	quality	date	ppg	state.name	lat	
0	Annandale	VA	100	7.075	high	2010-09-02	14.13	Virginia	38.830345	-7
1	Auburn	AL	60	28.300	high	2010-09-02	2.12	Alabama	32.578185	-8
2	Austin	TX	60	28.300	medium	2010-09-02	2.12	Texas	30.326374	-9
3	Belleville	IL	400	28.300	high	2010-09-02	14.13	Illinois	38.532311	-8
4	Boone	NC	55	3.540	high	2010-09-02	15.54	North Carolina	36.217052	-8

Exercise: The linear model I used in this chapter has the obvious drawback that it is linear, and there is no reason to expect prices to change linearly over time. We can add flexibility to the model by adding a quadratic term, as we did in Section 11.3.

Use a quadratic model to fit the time series of daily prices, and use the model to generate predictions. You will have to write a version of `RunLinearModel` that runs that quadratic model, but after that you should be able to reuse code from the chapter to generate predictions.

In [379...

```
def RunLinearModel(daily):
    """Runs a linear model of prices versus years.

    daily: DataFrame of daily prices

    returns: model, results
    """
    model = smf.ols('ppg ~ years', data=daily)
    results = model.fit()
    return model, results
```

In [381...

```
def GroupByDay(transactions, func=np.mean):
    grouped = transactions[['date', 'ppg']].groupby('date')
    daily = grouped.aggregate(func)

    daily['date'] = daily.index
    start = daily.date[0]
```

```

one_year = np.timedelta64(365, 'D')
daily['years'] = (daily.date - start) / one_year

return daily

```

```

In [383... def GroupByQualityAndDay(transactions):
    """Divides transactions by quality and computes mean daily price.

    transaction: DataFrame of transactions

    returns: map from quality to time series of ppg
    """
    groups = transactions.groupby('quality')
    dailies = {}
    for name, group in groups:
        dailies[name] = GroupByDay(group)

    return dailies

```

```

In [385... def PlotFittedValues(model, results, label=''):
    """Plots original data and fitted values.

    model: StatsModel model object
    results: StatsModel results object
    """
    years = model.exog[:, 1]
    values = model.endog
    thinkplot.Scatter(years, values, s=15, label=label)
    thinkplot.Plot(years, results.fittedvalues, label='model')

```

```

In [387... def PlotResidualPercentiles(model, results, index=1, num_bins=20):
    """Plots percentiles of the residuals.

    model: StatsModel model object
    results: StatsModel results object
    index: which exogenous variable to use
    num_bins: how many bins to divide the x-axis into
    """
    exog = model.exog[:, index]
    resid = results.resid.values
    df = pandas.DataFrame(dict(exog=exog, resid=resid))

    bins = np.linspace(np.min(exog), np.max(exog), num_bins)
    indices = np.digitize(exog, bins)
    groups = df.groupby(indices)

    means = [group.exog.mean() for _, group in groups][1:-1]
    cdfs = [thinkstats2.Cdf(group.resid) for _, group in groups][1:-1]

```

```
thinkplot.PrePlot(3)
for percent in [75, 50, 25]:
    percentiles = [cdf.Percentile(percent) for cdf in cdfs]
    label = '%dth' % percent
    thinkplot.Plot(means, percentiles, label=label)
```

```
In [389... def PlotLinearModel(daily, name):
    """Plots a linear fit to a sequence of prices, and the residuals.

    daily: DataFrame of daily prices
    name: string
    """

    model, results = RunLinearModel(daily)
    PlotFittedValues(model, results, label=name)
    thinkplot.Save(root='timeseries2',
                    title='fitted values',
                    xlabel='years',
                    xlim=[-0.1, 3.8],
                    ylabel='price per gram ($)',
                    formats=FORMATS)

    PlotResidualPercentiles(model, results)
    thinkplot.Save(root='timeseries3',
                    title='residuals',
                    xlabel='years',
                    ylabel='price per gram ($)',
                    formats=FORMATS)
```

```
In [391... def GenerateSimplePrediction(results, years):
    """Generates a simple prediction.

    results: results object
    years: sequence of times (in years) to make predictions for

    returns: sequence of predicted values
    """

    n = len(years)
    inter = np.ones(n)
    d = dict(Intercept=inter, years=years, years2=years**2)
    predict_df = pd.DataFrame(d)
    predict = results.predict(predict_df)
    return predict
```

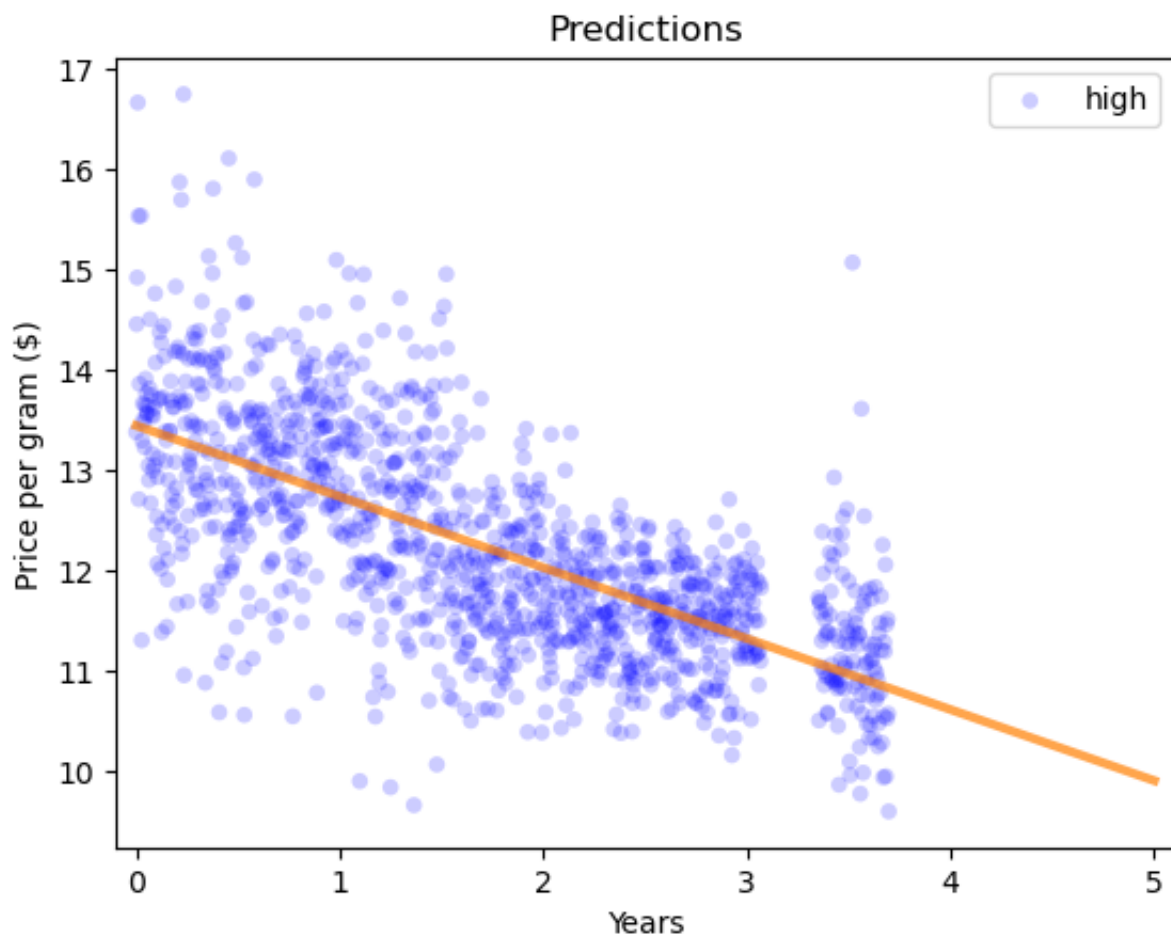
```
In [393... def PlotSimplePrediction(results, years):
    predict = GenerateSimplePrediction(results, years)

    thinkplot.Scatter(daily.years, daily.ppg, alpha=0.2, label=name)
    thinkplot.plot(years, predict, color="#ff7f00")
    xlim = years[0] - 0.1, years[-1] + 0.1
```

```
thinkplot.Config(
    title="Predictions",
    xlabel="Years",
    xlim=xlim,
    ylabel="Price per gram ($)",
    loc="upper right",
)
```

```
In [395... name = "high"
daily = dailies[name]

_, results = RunLinearModel(daily)
years = np.linspace(0, 5, 101)
PlotSimplePrediction(results, years)
```



```
In [397... def RunQuadraticModel(daily):
    """Runs a linear model of prices versus years.

    daily: DataFrame of daily prices

    returns: model, results
    """
```

```

daily["years2"] = daily.years**2
model = smf.ols("ppg ~ years + years2", data=daily)
results = model.fit()
return model, results

```

```

In [399... def SimulateResults(daily, iters=101, func=RunLinearModel):
    """Run simulations based on resampling residuals.

    daily: DataFrame of daily prices
    iters: number of simulations
    func: function that fits a model to the data

    returns: list of result objects
    """
    _, results = func(daily)
    fake = daily.copy()

    result_seq = []
    for _ in range(iters):
        fake.ppg = results.fittedvalues + thinkstats2.Resample(results.resid,
            _, fake_results = func(fake)
        result_seq.append(fake_results)

    return result_seq

```

```

In [401... def GeneratePredictions(result_seq, years, add_resid=False):
    """Generates an array of predicted values from a list of model results.

    When add_resid is False, predictions represent sampling error only.

    When add_resid is True, they also include residual error (which is
    more relevant to prediction).

    result_seq: list of model results
    years: sequence of times (in years) to make predictions for
    add_resid: boolean, whether to add in resampled residuals

    returns: sequence of predictions
    """
    n = len(years)
    d = dict(Intercept=np.ones(n), years=years, years2=years**2)
    predict_df = pandas.DataFrame(d)

    predict_seq = []
    for fake_results in result_seq:
        predict = fake_results.predict(predict_df)
        if add_resid:
            predict += thinkstats2.Resample(fake_results.resid, n)
        predict_seq.append(predict)

```

```
return predict_seq
```

```
In [403... def PlotPredictions(daily, years, iters=101, percent=90, func=RunLinearModel)
    """Plots predictions.

    daily: DataFrame of daily prices
    years: sequence of times (in years) to make predictions for
    iters: number of simulations
    percent: what percentile range to show
    func: function that fits a model to the data
    """

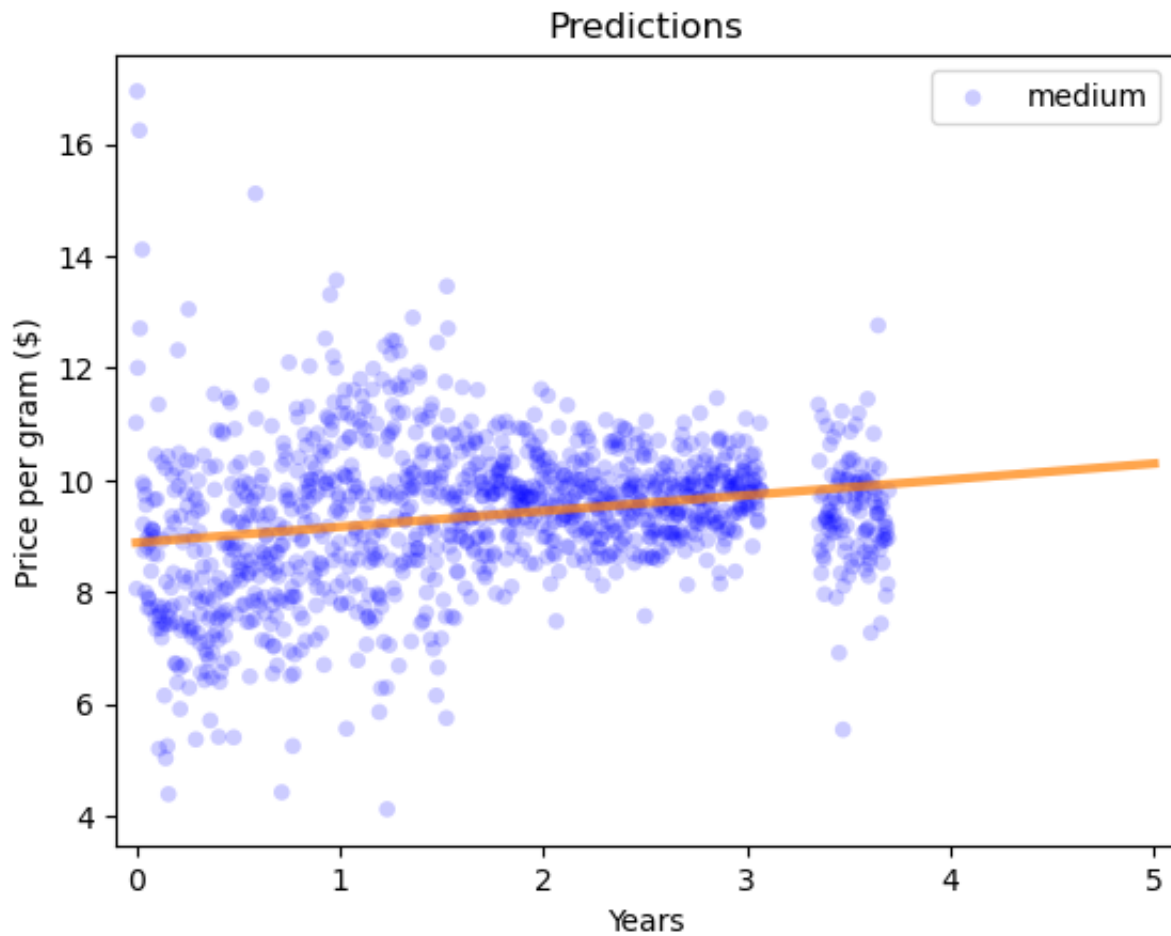
    result_seq = SimulateResults(daily, iters=iters, func=func)
    p = (100 - percent) / 2
    percents = p, 100 - p

    predict_seq = GeneratePredictions(result_seq, years, add_resid=True)
    low, high = thinkstats2.PercentileRows(predict_seq, percents)
    thinkplot.FillBetween(years, low, high, alpha=0.3, color="gray")

    predict_seq = GeneratePredictions(result_seq, years, add_resid=False)
    low, high = thinkstats2.PercentileRows(predict_seq, percents)
    thinkplot.FillBetween(years, low, high, alpha=0.5, color="gray")
```

```
In [405... name = "medium"
daily = dailies[name]

_, results = RunLinearModel(daily)
years = np.linspace(0, 5, 101)
PlotSimplePrediction(results, years)
```



```
In [407... def PlotFittedValues(model, results, label=''):
    """Plots original data and fitted values.

    model: StatsModel model object
    results: StatsModel results object
    """
    years = model.exog[:, 1]
    values = model.endog
    thinkplot.Scatter(years, values, s=15, label=label)
    thinkplot.Plot(years, results.fittedvalues, label='model')
```

```
In [409... # Solution

name = "high"
daily = dailies[name]

model, results = RunQuadraticModel(daily)
results.summary()
```

Out [409...

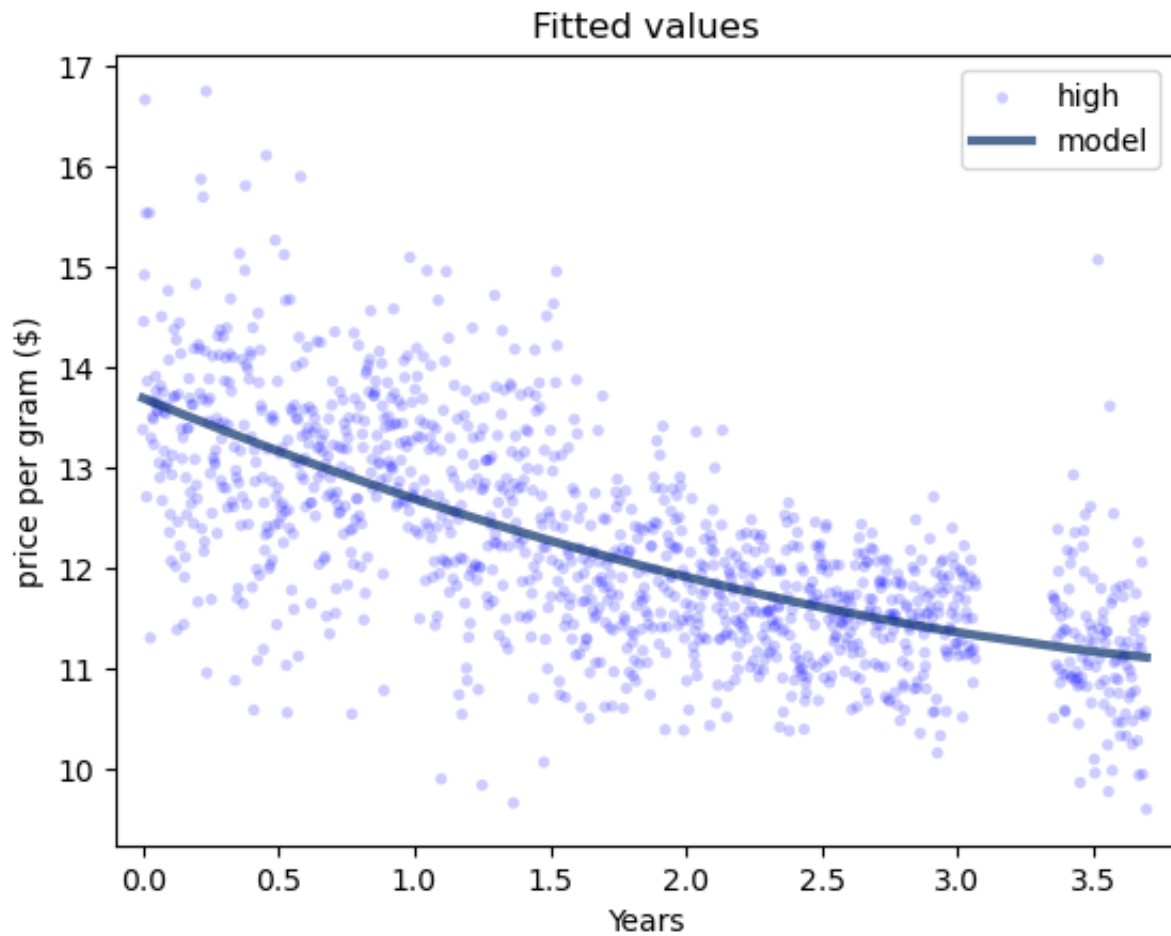
OLS Regression Results						
Dep. Variable:		ppg		R-squared:		0.455
Model:		OLS		Adj. R-squared:		0.454
Method:		Least Squares		F-statistic:		517.5
Date:		Sun, 03 Nov 2024		Prob (F-statistic):		4.57e-164
Time:		15:49:05		Log-Likelihood:		-1497.4
No. Observations:		1241		AIC:		3001.
Df Residuals:		1238		BIC:		3016.
Df Model:		2				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
Intercept	13.6980	0.067	205.757	0.000	13.567	13.829
years	-1.1164	0.084	-13.326	0.000	-1.281	-0.952
years2	0.1131	0.022	5.060	0.000	0.069	0.157
Omnibus:		49.112	Durbin-Watson:		1.885	
Prob(Omnibus):		0.000	Jarque-Bera (JB):		113.885	
Skew:		0.199	Prob(JB):		1.86e-25	
Kurtosis:		4.430	Cond. No.		27.5	

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

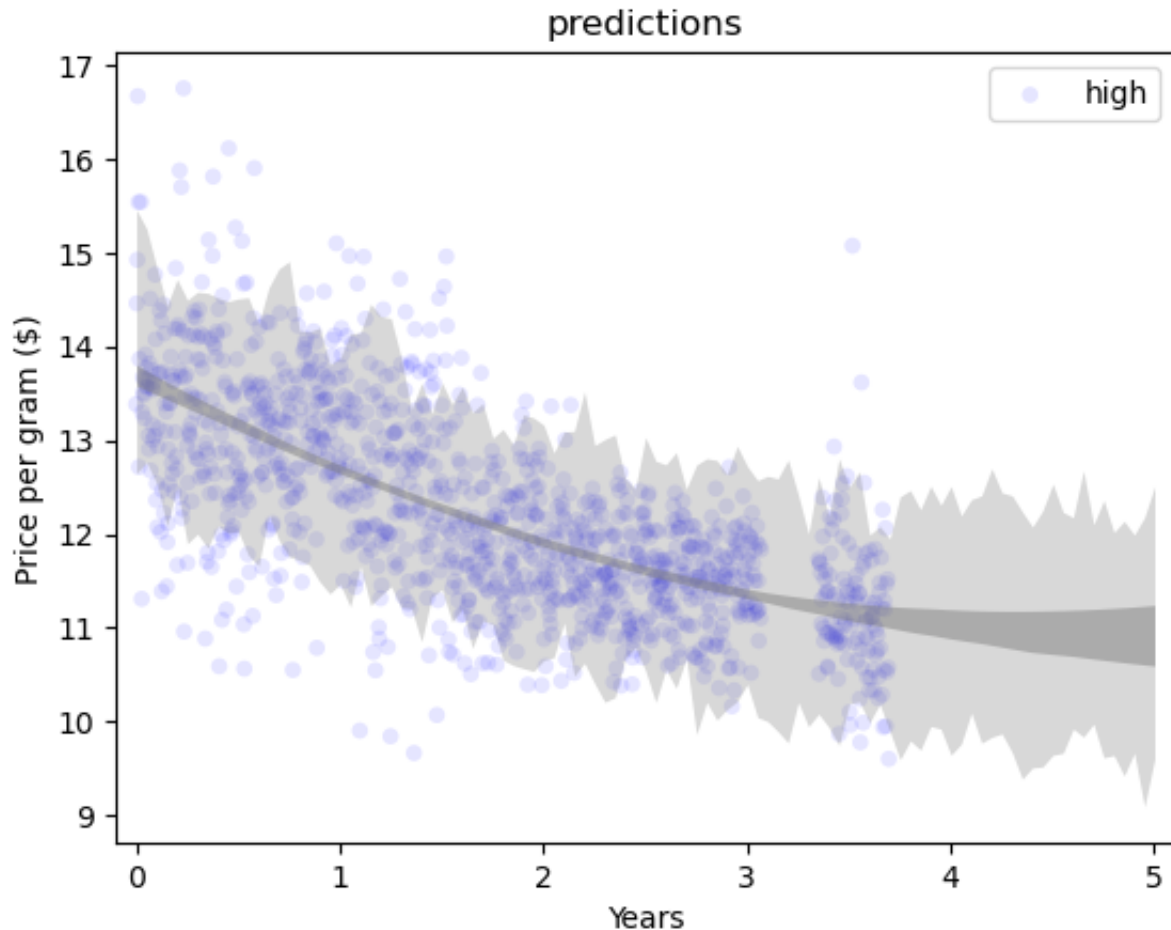
```
In [411... # Solution

PlotFittedValues(model, results, label=name)
thinkplot.Config(
    title="Fitted values", xlabel="Years", xlim=[-0.1, 3.8], ylabel="price p
)
```



In [413... *# Solution*

```
years = np.linspace(0, 5, 101)
thinkplot.Scatter(daily.years, daily.ppg, alpha=0.1, label=name)
PlotPredictions(daily, years, func=RunQuadraticModel)
thinkplot.Config(
    title="predictions",
    xlabel="Years",
    xlim=[years[0] - 0.1, years[-1] + 0.1],
    ylabel="Price per gram ($)",
)
```



Exercise: Write a definition for a class named `SerialCorrelationTest` that extends `HypothesisTest` from Section 9.2. It should take a series and a lag as data, compute the serial correlation of the series with the given lag, and then compute the p-value of the observed correlation.

Use this class to test whether the serial correlation in raw price data is statistically significant. Also test the residuals of the linear model and (if you did the previous exercise), the quadratic model.

```
In [416... def SerialCorr(series, lag=1):
    xs = series[lag:]
    ys = series.shift(lag)[lag:]
    corr = thinkstats2.Corr(xs, ys)
    return corr
```

```
In [418... # Solution

class SerialCorrelationTest(thinkstats2.HypothesisTest):
    """Tests serial correlations by permutation."""
```

```

def TestStatistic(self, data):
    """Computes the test statistic.

    data: tuple of xs and ys
    """
    series, lag = data
    test_stat = abs(SerialCorr(series, lag))
    return test_stat

def RunModel(self):
    """Run the model of the null hypothesis.

    returns: simulated data
    """
    series, lag = self.data
    permutation = series.reindex(np.random.permutation(series.index))
    return permutation, lag

```

```

In [420... # Solution

# test the correlation between consecutive prices

#name = "high"
daily = dailies["high"]

series = daily.ppg
test = SerialCorrelationTest((series, 1))
pvalue = test.PValue()
print(test.actual, pvalue)

```

0.48522937619473755 0.0

```

In [422... # Solution

# test for serial correlation in residuals of the linear model

_, results = RunLinearModel(daily)
series = results.resid
test = SerialCorrelationTest((series, 1))
pvalue = test.PValue()
print(test.actual, pvalue)

```

0.07570473767506251 0.009

```

In [424... # Solution

# test for serial correlation in residuals of the quadratic model

_, results = RunQuadraticModel(daily)

```

```
series = results.resid  
test = SerialCorrelationTest((series, 1))  
pvalue = test.PValue()  
print(test.actual, pvalue)
```

0.05607308161289923 0.044

In []: